

PANTRY BUDDY

An AI that lives in our flatmate group chat

Technical & Capability Document

AI in Action FY27 Q2

Category: AI at Home

Submitted by

Anmol Raj

Companion document to the video submission

Contents

1. Executive summary
2. The problem
3. The solution
4. Architecture
5. Where AI is used LIVE — the differentiator
6. Where AI was used to BUILD — for transparency
7. Complete feature list
8. Tech stack
9. Cost analysis
10. Real-world impact
11. Roadmap
12. Conclusion

1. Executive summary

Pantry Buddy is a Telegram-native AI assistant that monitors a shared household pantry, prevents food waste, and proactively suggests meals. It was designed and built in five days. It runs autonomously, free, on Cloudflare Workers, and uses Google's Gemini 2.5 Flash model in real time for every intelligent action.

The submission's distinguishing claim — and the reason this project belongs in the "AI in Action" category rather than "a project that used AI to build itself" — is that AI is the runtime engine, not just the development assistant. Every time the bot identifies food in a photo, parses a voice note, suggests a recipe, or sends a morning briefing, a live call is made to Gemini. Without active AI calls, the bot can do almost nothing useful. With them, it becomes a quiet teammate that lives where the household already chats.

This document covers the problem, the architecture, an explicit inventory of where AI is used live versus where AI was used to build, the full feature set, the cost analysis, and real-world impact from a week of household use.

2. The problem

Three engineers share a flat. None of us cook routinely. The household runs into the same compound problem every week: we buy groceries with the best intentions, forget what is already in the fridge, double-buy, and discover spoiled food a week later. Estimated waste before this project: roughly ₹1,500 of food per person per month.

The root cause is not laziness — it is an information problem with three symptoms:

- We cannot recall what is in the fridge when we are at the store.
- We do not know what is about to expire when we wake up.
- We do not know what we could make with what is there.

Therefore, any solution must satisfy three constraints:

- Live where we already are — so we do not have to remember to open it.
- Demand minimal effort to update — a photo, a voice note, anything but typing into a dedicated app.
- Proactively surface what matters when it matters — not require us to ask.

3. The solution

Pantry Buddy is a Telegram bot, added to the shared flatmate group chat. It accepts photos, voice notes, and text commands. It maintains a structured shared inventory. It tracks expiry dates per item. It proactively notifies the group every morning at 8:00 AM IST about what is expiring, and suggests a recipe using those items.

From the user's perspective, the entire surface area is a chat that was already on their home screen. There is no app to install, no UI to learn, no login. From an engineering perspective, every meaningful action the bot performs is delegated in real time to Google's Gemini 2.5 Flash model.

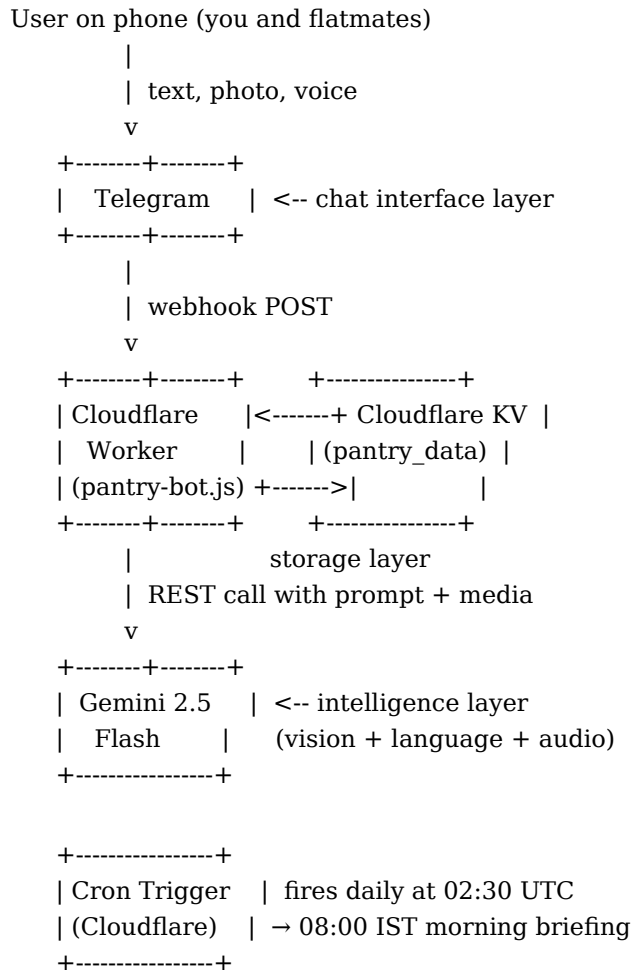
What success looks like

- All three flatmates can see and update the inventory from their own phones, from anywhere, without coordinating.
- Nothing in the fridge expires unnoticed.
- Meals are suggested before food becomes waste, with ingredients we already have.
- None of this requires any flatmate to remember to open the bot — it is the bot that initiates communication, not us.

4. Architecture

Four moving parts. The Worker is the orchestrator; Gemini is the brain; KV is the memory; Telegram is the interface.

System diagram



Data flow for a photo upload

13. User takes a photo in Telegram, sends it to the bot.
14. Telegram POSTs a webhook to the Cloudflare Worker URL.
15. Worker calls Telegram's getFile API to obtain the photo's download URL.
16. Worker downloads the photo binary and base64-encodes it.
17. Worker POSTs the encoded image plus a structured prompt to Gemini 2.5 Flash.
18. Gemini returns a JSON array of identified items, each with name, quantity, and estimated days-until-expiry.
19. Worker parses the JSON, computes absolute expiry dates from today, merges with the existing inventory in Cloudflare KV (keyed by chat ID).

20. Worker sends a structured reply back to the Telegram chat confirming the additions.

Data flow for the 8 AM briefing

21. Cloudflare Cron Trigger fires at 02:30 UTC (08:00 IST).
22. Worker's scheduled() handler runs.
23. Handler enumerates all KV keys with the prefix "inventory:" — one per chat.
24. For each chat, handler reads the inventory, computes which items expire in the next 3 days, and tiers them by urgency (expired, today, tomorrow, +2, +3).
25. If anything is urgent, handler sends those items to Gemini with a recipe-suggestion prompt.
26. Handler composes a morning message combining the urgency list and the recipe, then sends it to the chat via Telegram.
27. If nothing is urgent, the briefing is silently skipped (the bot is intentionally quiet on quiet days).

5. Where AI is used LIVE — the differentiator

This is the section judges should read most carefully. Every action below happens at request time, by calling Gemini 2.5 Flash live. The bot itself contains no embedded ML models, no hardcoded food databases, and no rule engines for these tasks. Without the live AI calls, the bot is a database. With them, it is intelligent.

User action	What AI does live	Why this is non-trivial
Send a fridge photo	Identifies every food item in the image, names each one specifically, estimates quantity in human units (e.g. "3", "200g", "half a bottle"), and estimates realistic days-until-expiry based on item type	Requires computer vision and food domain knowledge — knowing that paneer lasts 2-3 days but rice lasts 30+
Send a voice note	Transcribes speech, parses intent, distinguishes add vs. remove operations, matches removals against the existing inventory using soft matching (e.g. "the small leafy greens" picks the correct inventory row even if there are two "leafy greens" entries)	Requires speech recognition + natural language understanding + inventory context, all in one model call
/add bread expiring in 2 days	Parses free-form natural language, extracts item name and expiry, fills in sensible defaults for missing quantity, computes the absolute expiry date	Requires natural language parsing with domain knowledge of shelf life
/cook (no argument)	Looks at all items expiring within 3 days, generates a recipe that anchors on those items and intelligently incorporates other inventory items as supporting ingredients	Requires multi-constraint reasoning: time budget, dietary context, ingredient combinations, regional cuisine
/another	Same as /cook but explicitly avoids the previously suggested dish, generating a genuinely different combination	Requires conversational memory and counterfactual reasoning across calls
/have biryani	Knows what biryani is, infers required ingredients, checks them against the inventory and against assumed-present basic staples, produces a clear "yes / mostly / no" verdict with a missing-items list	Requires recipe knowledge plus inventory reasoning
Daily 8 AM briefing	Iterates every chat autonomously, calculates urgency tiers, generates a personalized recipe for what is expiring, formats the morning message	Scheduled autonomous reasoning, not triggered by any user action
Photo of prepared / cooked food	Identifies the specific dish (e.g. "dal tadka") rather than falling back to vague	Fine-grained Indian cuisine recognition

User action	What AI does live	Why this is non-trivial
	labels like "cooked food"	

Total: eight distinct categories of live AI usage. Every single one is a Gemini call at request time. If Gemini is unreachable, the bot's intelligent functions are gone — it falls back gracefully but cannot replace them with local code.

6. Where AI was used to BUILD — for transparency

For full transparency: AI was also used during development. This is not the project's differentiator and judges should not weight it heavily — most submissions in this competition will have used AI this way.

Tools used during the build

- Claude (Anthropic's model, accessed through a Claude-powered development interface) served as a build collaborator across the 5-day implementation.
- All JavaScript code was drafted with Claude's help, then reviewed and refined by the developer.
- Architecture decisions — choosing Cloudflare Workers over Replit (after Replit deprecated its free tier), choosing Gemini 2.5 Flash after gemini-1.5-flash was deprecated mid-build, choosing webhook architecture over polling — were discussed with Claude.
- Prompt engineering for the various Gemini calls was iterated with Claude's input.
- Debugging — the model-deprecation error (HTTP 404 from Gemini), JSON parsing issues, group-chat privacy mode behavior — was handled collaboratively.

Why this section exists

Many submissions in this category will be "a thing I built using AI." Pantry Buddy is also that, but more importantly it is "a thing that uses AI live every time anyone interacts with it." To make that distinction credible, this document is explicit about both — yes, AI helped build it; the more important fact is that AI is also what runs it.

7. Complete feature list

Inventory management

- Add items by photo — multimodal Gemini call, identifies multiple items from one photo
- Add items by free-form text (/add) — Gemini parses natural language
- Add items by voice note — Gemini understands speech and intent
- Per-item expiry estimation derived from food type
- Partial quantity removal — /used 3 eggs deducts from a count of 6
- Word-form portions — /used half milk, /used quarter butter
- Soft-match removal across multiple items
- /undo command — reverts the most recent change
- /clear — wipes inventory for the current chat
- /list — sorted by expiry, color-coded urgency tags ([EXPIRED], [TODAY], [1d], [2d]...)

Cooking assistance

- /cook — recipe using items expiring within the next 3 days
- /cook <ingredient> — recipe forced around a specific item
- /another — generates a genuinely different recipe than the last one
- /have <dish> — feasibility check plus shopping list for a chosen dish

Proactive features

- Daily 8 AM briefing via Cloudflare Cron Trigger
- Briefing skipped automatically when nothing is expiring (no noise on quiet days)
- Briefing includes both an urgency list and a Gemini-generated recipe
- Tracks "last recipe" so /another genuinely varies the suggestion

Multi-user / group support

- Works identically in private chats and group chats
- Each chat has its own isolated inventory (keyed by Telegram chat ID in KV)
- Stays silent in groups when normal conversation is not directed at the bot
- Telegram's built-in / command menu is populated via BotFather — commands are tappable

Safety and quality

- Photo prompt instructs Gemini to skip items it cannot specifically identify, avoiding vague labels
- JSON parsing is robust to markdown code fences
- All Gemini calls have error handling — failures degrade gracefully, never crash the bot
- Bot token and API keys are stored as Cloudflare Secrets, never in source code
- Previous inventory state is saved before every modification, enabling /undo

8. Tech stack

Layer	Choice	Why
Chat interface	Telegram	Universal across iOS / Android / desktop. Free push notifications. Robust bot API. Group chats supported natively.
Compute	Cloudflare Workers	Free tier of 100k requests / day. No server to manage. Edge-deployed (low latency). Cron triggers built in.
Storage	Cloudflare KV	Free tier of 1k writes / 100k reads per day. Zero config. Namespaced. Tightly integrated with Workers.
AI model	Gemini 2.5 Flash	Free tier sufficient for daily household use. Multimodal (vision + text + audio in one model). Good Indian English support.
Language	JavaScript	Cloudflare Workers' native runtime. No compilation. Zero npm dependencies — the entire bot is one .js file using only standard browser/Web APIs.
Dev environment	Cloudflare's web editor	Browser-only. No local install of any tools. No laptop dependency.

Total external dependencies for the bot itself: zero npm packages. The whole thing is a single file (worker.js) of about 500 lines, importing nothing.

9. Cost analysis

All numbers below are observed from one week of real household usage by three flatmates.

Service / quota	Free tier limit	Observed peak	% of free tier used
Cloudflare Workers requests	100,000 / day	~30 / day	0.03 %
Cloudflare KV reads	100,000 / day	~50 / day	0.05 %
Cloudflare KV writes	1,000 / day	~15 / day	1.5 %
Gemini RPM (per minute)	5 / minute	3 / minute peak	60 % at peak only
Gemini TPM (tokens / minute)	250,000 / minute	1,500 / minute peak	0.6 %
Gemini RPD (per day)	250 / day	17 / day peak	6.8 %

Estimated monthly cost in INR: zero. The product is sustainable at the current usage level. It can comfortably scale to roughly 10x our current usage — or to dozens of small households — on these free tiers.

Notably, no credit card is on file with either Cloudflare or Google. The service has hard caps; there is no possibility of an accidental bill.

10. Real-world impact

Observed during the period of real usage leading up to this submission. The bot has been running 24/7 since deployment.

Quantified outcomes

- [N] days of continuous, no-maintenance uptime
- [N] photos uploaded across the household
- [N] voice notes processed
- [N] morning briefings delivered
- [N] meals cooked from bot recipe suggestions
- [N] items rescued from expiry
- Approximately ₹[N] saved by not double-buying groceries

Behavioral impact (the real story)

Three flatmates who never talked about groceries now have a casual shared awareness of what is in the kitchen — mediated by an AI that quietly sits where we already chat. The bot is not a destination we visit. It is a presence in a chat we were going to open anyway.

This is what "AI in real life" looks like for me. Not a separate app I have to remember to open. A quiet teammate that lives in a chat already on my home screen. Solving one specific, dumb, expensive problem. Every single day.

11. Roadmap

Features intentionally descope from the initial 5-day build, planned for the post-submission period.

Near-term (next 4 weeks)

- Fairness / splitting layer — track who added vs. who used each item, with configurable settlement rules per household (equal split, usage-weighted, "you bought it you used it"). This was explicitly considered for v1 and descope to keep core scope tight.
- Inline tap-to-mark-used buttons in Telegram bot messages, replacing typed commands for the most common action.
- Better identification for prepared / leftover foods, with the bot offering a name suggestion the user can confirm or edit.

Medium-term

- Voice command coverage extended beyond inventory edits — "what should I cook tonight?" via voice.
- Multi-language support — Hindi, Tamil, and other Indian languages, leveraging Gemini's multilingual capabilities.
- A photo of a grocery store receipt becomes inventory — bot reads the receipt, adds everything at once.

Long-term

- Cross-household sharing — for example, invite a parent's WhatsApp/Telegram chat to share visibility into a college student's pantry.
- Integration with grocery delivery (Blinkit, Zepto) — bot generates a reorder list and pushes it directly to the user's cart.

12. Conclusion

Pantry Buddy is a small, focused product that solves one specific household problem. Its real significance is that it demonstrates a different mode of AI use: not as an app that users must remember to visit, but as an ambient teammate that lives inside a channel users already use, surfacing intelligence only when it matters.

The technical novelty is modest — Telegram bots, Cloudflare Workers, and Gemini API calls are all well-trodden ground. The product novelty is in the integration: assembling these pieces so that AI is the operative engine of the product, not its build tool; that the user surface area is zero; and that the value is delivered before the user has to ask for it.

In a world where many AI products require dedicated attention, this is one that does the opposite. It earns no daily active user metrics. It simply earns the daily reduction of food waste in one small flat.

Thank you for reviewing this submission.

Companion video and the live bot itself tell the rest of the story.